

10 — Covering Indexes and Index-Only Scans

"A covering index is the ultimate optimization — the heap never needs to be touched."

Table of Contents

1. [Learning Objectives](#)
 2. [What is a Covering Index](#)
 3. [Index-Only Scan Mechanics](#)
 4. [ASCII Diagram: Index-Only Scan vs Index Scan](#)
 5. [The INCLUDE Clause](#)
 6. [INCLUDE vs Key Columns](#)
 7. [Visibility Map and Heap Fetches](#)
 8. [Creating Covering Indexes](#)
 9. [EXPLAIN Output](#)
 10. [Covering Indexes for COUNT and Aggregates](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Performance Considerations](#)
 14. [Interview Questions & Answers](#)
 15. [Exercises with Solutions](#)
 16. [Production Scenarios](#)
 17. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain what a covering index is and how it enables index-only scans
 - Use the `INCLUDE` clause to add non-key columns to an index
 - Describe the visibility map and how it affects index-only scan heap fetches
 - Measure the performance difference between an index scan and an index-only scan
 - Identify queries that can benefit from covering indexes
 - Design covering indexes that avoid key-order bloat from `INCLUDE` columns
-

What is a Covering Index

A **covering index** is an index that contains all columns referenced by a query — both the `WHERE`/`JOIN` columns (for lookup) and the `SELECT` columns (for data retrieval). When all required data is in the index, the heap pages never need to be read.

```
-- Query:
SELECT email, created_at FROM users WHERE user_id = 42;

-- Regular index on user_id:
-- Step 1: Index scan finds ctid for user_id=42
-- Step 2: HEAP FETCH – read heap page to get email and created_at

-- Covering index on user_id INCLUDE (email, created_at):
```

```
-- Step 1: Index scan finds user_id=42 AND reads email, created_at from index
-- Step 2: NO HEAP FETCH needed!
```

The result is called an **Index-Only Scan** — the query is answered entirely from index pages, which are typically much smaller and more likely to be cached.

Index-Only Scan Mechanics

For an index-only scan to succeed, two conditions must be met:

1. **All referenced columns are in the index** (either as key columns or INCLUDE columns)
2. **The heap page is visible to the current transaction** (visibility map check)

If condition 2 fails for some pages, PostgreSQL falls back to fetching those heap pages to verify visibility, but this is the minority case after VACUUM has run.

ASCII Diagram: Index-Only Scan vs Index Scan

Regular Index Scan (two storage accesses):

Query: `SELECT email FROM users WHERE user_id = 42`

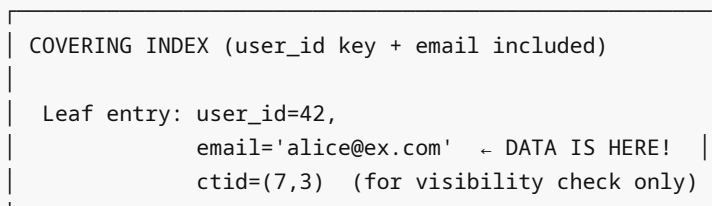


I/O: index pages + heap pages

Index-Only Scan (one storage access):

Query: `SELECT email FROM users WHERE user_id = 42`

Index: `CREATE INDEX ON users(user_id) INCLUDE (email)`



|
| Visibility map check
▼

VISIBILITY MAP for page 7
Page 7: all-visible = TRUE ✓

← cheap 1-bit check, no heap read

Return email='alice@ex.com' directly from index!
I/O: only index pages (no heap pages!)

The INCLUDE Clause

The `INCLUDE` clause (PostgreSQL 11+) adds **non-key columns** to index leaf pages. These columns are stored with each leaf entry but are NOT part of the sort key.

```
CREATE INDEX idx_users_lookup ON users(user_id)
INCLUDE (email, created_at, status);
```

Syntax

```
CREATE INDEX [CONCURRENTLY] index_name
ON table_name (key_col1, key_col2, ...)
INCLUDE (non_key_col1, non_key_col2, ...);
```

What Goes in INCLUDE

- Columns that appear in `SELECT` but not in `WHERE` / `JOIN` predicates
- Columns that don't need to be part of the sort order
- Large columns (text, JSON, bytea) — putting them in key would bloat the tree

INCLUDE vs Key Columns

Aspect	Key Column	INCLUDE Column
Part of sort order	Yes	No
In internal pages	Yes	No (leaf only)
Can be used in WHERE/ORDER BY	Yes	Not directly
Used for range scans	Yes	No
Added to internal node size	Yes	No
Unique constraint checks	Yes	No

Why INCLUDE Instead of Adding to Key?

Consider `CREATE INDEX ON orders(customer_id, total_amount)` vs `CREATE INDEX ON orders(customer_id) INCLUDE (total_amount)` :

Adding `total_amount` as KEY column:

- Stored in internal pages (increases tree depth/fanout)
- Used for sort order (unnecessary if never filtered on)
- Internal pages hold `(customer_id, total_amount)` → bigger pages → less entries per page

Adding `total_amount` as INCLUDE column:

- Only in leaf pages (internal pages stay small)
- Internal pages hold only `(customer_id)` → more entries per page → shallower tree
- Leaf pages are bigger, but internal pages (navigated for every lookup) are compact

The `INCLUDE` approach keeps the tree structure efficient while still enabling index-only scans.

Visibility Map and Heap Fetches

PostgreSQL's **visibility map** is a 2-bit-per-page bitmap that tracks:

- `all-visible` : all tuples on this page are visible to all transactions
- `all-frozen` : all tuples are frozen (never needs VACUUM)

How Index-Only Scan Uses the Visibility Map

For each `ctid` returned by the index:

1. Check visibility map for the heap page
2. If `all-visible = TRUE`:
 - Return data from index (no heap fetch)
3. If `all-visible = FALSE`:
 - Fetch heap page to check individual tuple visibility
 - Return matching tuples

Fresh Tables vs. Vacuumed Tables

```
-- Immediately after data load (no VACUUM yet):
-- All visibility map bits = 0 (not all-visible)
-- Index-only scan falls back to heap fetches for every row!
-- EXPLAIN shows: Heap Fetches: 500000 (bad)

-- After VACUUM (or autovacuum):
-- Visibility map bits set to all-visible for stable pages
-- True index-only scan:
-- EXPLAIN shows: Heap Fetches: 0 (perfect)

-- Force vacuum to enable index-only scans:
VACUUM users;
-- or for the whole database:
VACUUM;
```

Check Current Visibility Map Status

```
-- Pages that need vacuum (not all-visible)
SELECT
    relname,
    n_dead_tup,
    n_live_tup,
    last_vacuum,
    last_autovacuum
FROM pg_stat_user_tables
WHERE relname = 'users';
```

Creating Covering Indexes

```
-- Covering index for a typical user lookup
CREATE INDEX CONCURRENTLY idx_users_id_covering
ON users(user_id)
INCLUDE (email, display_name, avatar_url, created_at);

-- Covering index for order listing
CREATE INDEX CONCURRENTLY idx_orders_cust_covering
ON orders(customer_id, created_at DESC)
INCLUDE (id, status, total_amount, item_count);

-- Covering partial index (combines three features)
CREATE INDEX CONCURRENTLY idx_active_users_email_covering
ON users(lower(email))          -- expression key
INCLUDE (id, display_name)      -- included columns
WHERE deleted_at IS NULL;      -- partial predicate

-- Covering index for a join
CREATE INDEX CONCURRENTLY idx_order_items_order_covering
ON order_items(order_id)
INCLUDE (product_id, quantity, unit_price);
-- Supports: SELECT product_id, quantity, unit_price FROM order_items WHERE order_id
-- = ?
-- Without heap fetch
```

EXPLAIN Output

Index-Only Scan (optimal)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT email, created_at FROM users WHERE user_id = 42;
-- Index: users(user_id) INCLUDE (email, created_at)
```

```
Index Only Scan using idx_users_id_covering on users
  (cost=0.43..4.45 rows=1 width=40)
  (actual time=0.034..0.034 rows=1 loops=1)
Index Cond: (user_id = 42)
Heap Fetches: 0          ← Perfect! No heap reads
Buffers: shared hit=3
Planning Time: 0.145 ms
Execution Time: 0.056 ms
```

Index-Only Scan with Heap Fetches (needs VACUUM)

```
-- Same query, table hasn't been vacuumed recently
```

```
Index Only Scan using idx_users_id_covering on users
  (cost=0.43..4.45 rows=1 width=40)
  (actual time=0.089..0.123 rows=1 loops=1)
Index Cond: (user_id = 42)
Heap Fetches: 1          ← Heap fetch needed (visibility check failed)
Buffers: shared hit=4
```

Index Scan (no covering index)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT email FROM users WHERE user_id = 42;
-- Index: users(user_id) – no INCLUDE
```

```
Index Scan using idx_users_pkey on users
  (cost=0.43..8.45 rows=1 width=20)
  (actual time=0.045..0.056 rows=1 loops=1)
Index Cond: (user_id = 42)
Buffers: shared hit=4    ← Extra heap page read
Planning Time: 0.134 ms
Execution Time: 0.078 ms
```

Covering Indexes for COUNT and Aggregates

Covering indexes can dramatically speed up aggregate queries.

```
-- Covering index to speed up COUNT
CREATE INDEX CONCURRENTLY idx_orders_status_covering
ON orders(status)
INCLUDE (customer_id);

-- Count query – might use index-only scan
EXPLAIN SELECT status, COUNT(*) FROM orders GROUP BY status;

-- For pure COUNT(*) with no GROUP BY, a small partial index on any NOT NULL column
```

```
-- can be used for an index-only scan:
CREATE INDEX CONCURRENTLY idx_orders_count_opt ON orders(id)
WHERE id IS NOT NULL;
-- SELECT COUNT(*) FROM orders → Index Only Scan, much faster than Seq Scan
```

Range Aggregates

```
-- Covering index for time-range aggregation
CREATE INDEX CONCURRENTLY idx_orders_date_covering
ON orders(created_at)
INCLUDE (total_amount, status);

-- Query uses index-only scan for the aggregate:
SELECT SUM(total_amount), COUNT(*)
FROM orders
WHERE created_at BETWEEN '2024-06-01' AND '2024-06-30'
AND status = 'completed';
```

Common Mistakes

1. Adding too many columns to INCLUDE

```
-- OVERKILL: nearly the whole row in the index
CREATE INDEX ON orders(customer_id) INCLUDE (
    id, status, created_at, updated_at, total_amount, item_count,
    shipping_address, billing_address, notes, metadata
);
-- Index becomes as large as the heap – defeats the purpose
```

2. Adding KEY columns that should be INCLUDE columns

```
-- BAD: email is never used in WHERE/ORDER BY but added as a KEY column
CREATE INDEX ON users(user_id, email);
-- email inflates internal pages, reducing B-tree fanout

-- GOOD: email as INCLUDE only
CREATE INDEX ON users(user_id) INCLUDE (email);
```

3. Not running VACUUM after bulk loads

After a large INSERT/COPY, visibility map bits are not set. Index-only scans will still do heap fetches. Run `VACUUM table_name` after bulk loads to set visibility bits.

4. Expecting index-only scan without verifying visibility

```
-- EXPLAIN says "Index Only Scan" but you see Heap Fetches: 50000?
-- The table needs VACUUM.
```

```
VACUUM ANALYZE orders;
```

5. Using INCLUDE with a partial index and expecting all queries to work

A partial index with INCLUDE only works for queries that satisfy the partial predicate.

Best Practices

1. **Identify the most frequent, performance-critical read queries** and design covering indexes for them.
2. **Use INCLUDE for columns that appear in SELECT but not WHERE** — they don't need to be key columns.
3. **Keep INCLUDE columns selective** — don't add large TEXT or JSONB columns to INCLUDE unless they're critical for query performance.
4. **Run VACUUM after bulk loads** to set visibility map bits and enable true index-only scans.
5. **Monitor Heap Fetches in EXPLAIN** — if non-zero after VACUUM, the index may be partially correct or autovacuum settings need tuning.
6. **Combine with partial indexes** for optimal size: `CREATE INDEX ON hot_orders(id, customer_id) INCLUDE (total) WHERE status = 'active' .`
7. **Check index size** after adding INCLUDE columns:

```
SELECT pg_size_pretty(pg_relation_size('idx_name'));
```

Performance Considerations

Index-Only Scan Speedup

Typical speedups from covering indexes:

- Lookup queries: 2–5× faster (eliminating heap page reads)
- Aggregate queries on large ranges: 5–20× faster (no random I/O to heap)
- Hot-path API queries: often the difference between 1ms and 0.1ms

Memory Efficiency

Index pages are typically much smaller than heap pages for the same data:

```
Heap page (8 KB): 100 rows × 200 bytes avg
Index leaf page (8 KB): 200 entries × 40 bytes avg
Index-only scan reads fewer pages total → better cache utilization
```

VACUUM Frequency

For tables with high update/delete rates, visibility map bits are frequently cleared. Index-only scan effectiveness degrades until the next VACUUM runs. Consider:


```
-- Increase autovacuum frequency for tables with covering indexes
ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.01, -- vacuum after 1% rows changed
    autovacuum_analyze_scale_factor = 0.005 -- analyze after 0.5% changed
);
```

Interview Questions & Answers

Q1. What is a covering index and what performance benefit does it provide?

A: A covering index is an index that contains all columns required to answer a query — both the lookup key columns and the data columns that the query returns (via SELECT). When a covering index is present and the visibility map confirms all heap pages are visible, PostgreSQL performs an Index Only Scan: it answers the query entirely from index pages without reading any heap pages. This eliminates the random I/O to heap pages, which is the most expensive part of an Index Scan. The result is typically 2–5× faster for point lookups and can be 10–20× faster for aggregate queries on large ranges.

Q2. What is the difference between a key column and an INCLUDE column?

A: Key columns are part of the B-tree sort key — they are stored at every level of the tree (root, internal, leaf) and determine the sort order. They can be used in WHERE, JOIN, and ORDER BY predicates for the index scan. INCLUDE columns (PostgreSQL 11+) are only stored in leaf pages — they are not part of the sort key and cannot be used to narrow the index scan. INCLUDE columns exist solely to make the leaf entry self-sufficient for covering the SELECT columns. Because INCLUDE columns are not in internal pages, they don't affect the tree's fanout or depth, keeping the tree structure compact.

Q3. Why might an Index Only Scan show Heap Fetches > 0 in EXPLAIN ANALYZE?

A: Each ctid returned by the index is checked against the visibility map. If the corresponding heap page is not marked all-visible in the visibility map (e.g., because it has recently been modified or the table hasn't been VACUUMed recently), PostgreSQL must read the actual heap page to verify that the tuple is visible to the current transaction. This produces a heap fetch. After VACUUM runs and sets all-visible bits for stable pages, Heap Fetches drops to 0. For very high-churn tables, heap fetches may persist even with frequent VACUUM.

Q4. How do you design a covering index for a SELECT with multiple returned columns?

A: Identify: (a) the WHERE/JOIN columns → these become key columns in the index, (b) the remaining SELECT columns → these go into the INCLUDE clause. For example, for `SELECT user_id, email, created_at FROM users WHERE tenant_id = ? AND role = 'admin'`: Key columns: `(tenant_id, role)` — used for the range scan. INCLUDE columns: `(user_id, email, created_at)` — returned by SELECT, not used for scan. Result: `CREATE INDEX ON users(tenant_id, role) INCLUDE (user_id, email, created_at) .`

Q5. What is the visibility map and why does it matter for index-only scans?

A: The visibility map is a 1-bit-per-heap-page (actually 2-bit in recent PG versions) bitmap that records whether all tuples on each page are visible to all transactions. VACUUM sets the all-visible bit after cleaning dead tuples from a page. For index-only scans, PostgreSQL uses the visibility map to avoid reading heap pages — if the bit is set, the data in the index is definitively what's in the heap, so the heap page read is skipped. Without the

visibility map, every index-only scan would still need to fetch heap pages to check visibility, making it equivalent to a regular index scan.

Exercises with Solutions

Exercise 1

The following query runs on an `orders` table with 50M rows and is called 10,000 times/second. Design the optimal covering index.

```
SELECT id, status, total_amount
FROM orders
WHERE customer_id = $1
ORDER BY created_at DESC
LIMIT 10;
```

Solution:

```
-- Analysis:
-- WHERE: customer_id = $1 → key column #1 (equality)
-- ORDER BY: created_at DESC → key column #2 (provides sort order)
-- SELECT: id, status, total_amount → INCLUDE columns

CREATE INDEX CONCURRENTLY idx_orders_cust_date_covering
ON orders(customer_id, created_at DESC)
INCLUDE (id, status, total_amount);

-- Expected plan:
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, status, total_amount
FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC
LIMIT 10;

-- Expected output:
-- Index Only Scan using idx_orders_cust_date_covering on orders
--   Index Cond: (customer_id = 42)
--   Heap Fetches: 0
-- → No heap reads, no sort, stops after 10 entries
```

Exercise 2

You have a `products` table and the following query:

```
SELECT COUNT(*), AVG(price) FROM products WHERE category_id = 5 AND is_active =
TRUE;
```

Design a covering index and verify with EXPLAIN.

Solution:

```
CREATE INDEX CONCURRENTLY idx_products_cat_active_cov
ON products(category_id, is_active)
INCLUDE (price);

VACUUM products; -- ensure visibility map is current

EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*), AVG(price)
FROM products
WHERE category_id = 5 AND is_active = TRUE;

-- Expected: Index Only Scan, Heap Fetches: 0
-- price is read from the index leaf pages → no heap access
```

Production Scenarios

Scenario 1: High-Frequency User Profile Lookups

A social network API serves 1 million user profile requests per second. The query is: `SELECT user_id, display_name, avatar_url, bio FROM users WHERE username = ?`

```
-- Before: Index scan with heap fetch per request
-- After: Covering index

CREATE INDEX CONCURRENTLY idx_users_username_covering
ON users(lower(username)) -- expression key for case-insensitive
INCLUDE (user_id, display_name, avatar_url, bio)
WHERE is_active = TRUE; -- partial: only active users

VACUUM users;

-- Result: 1M req/s, each 0.05ms → index-only scan, no heap I/O
-- Server can now handle 2M req/s with same hardware
```

Scenario 2: Analytics Dashboard Summary

A dashboard shows 30-day order stats per customer. Runs every page load.

```
CREATE INDEX CONCURRENTLY idx_orders_dash
ON orders(customer_id, created_at)
INCLUDE (status, total_amount)
WHERE created_at > '2024-01-01'; -- only recent data

-- Dashboard query:
SELECT
    COUNT(*) AS order_count,
    SUM(total_amount) AS revenue,
    COUNT(*) FILTER (WHERE status = 'completed') AS completed
```

```
FROM orders
WHERE customer_id = $1
      AND created_at > NOW() - INTERVAL '30 days';
-- Index Only Scan: reads index leaf entries only → sub-millisecond
```

Cross-References

- [02 btree indexes.md](#) — B-tree structure
- [09 composite indexes.md](#) — Composite index key design
- [08 partial indexes.md](#) — Partial index predicates
- [../08 Query Optimization/03 scan types.md](#) — Index Only Scan details
- [12 index maintenance.md](#) — VACUUM and visibility map